

Refonte de Kraftwerk

ou

« Construire le moteur post-collecte de demain »

Pilotée par l'IA
22/10/2025



Sommaire



01

Démarche et objectifs



02

Architecture cible



03

Roadmap

01

Démarche et objectifs

Pourquoi et comment on construit la cible

Les problèmes actuels

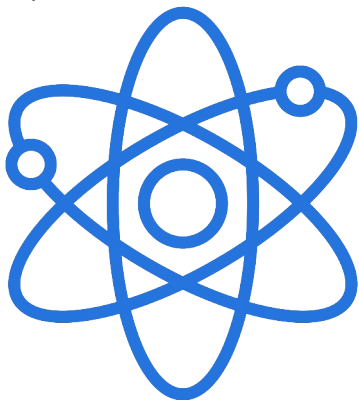
Complexité technique

Conception sur une cible différente de la cible actuelle (fichiers plats versus base par exemple)

Bugs récurrents

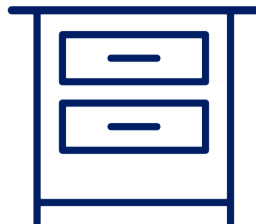
Run coûteux car des parties de codes qui ne font plus sens

Pertinence de l'outil pour des opérations « simples » (mise en forme des reporting data)



Pas de cohérence aval

Calculs basés sur des agrégats (sum, first, filter...) non recalculés



Automatisation et passage par « lot / partitionId »

QuestionnaireModelId n'est plus la référence

Période de transition avec des enquêtes automatisées et d'autres non donc complexité

Surcharge mémoire

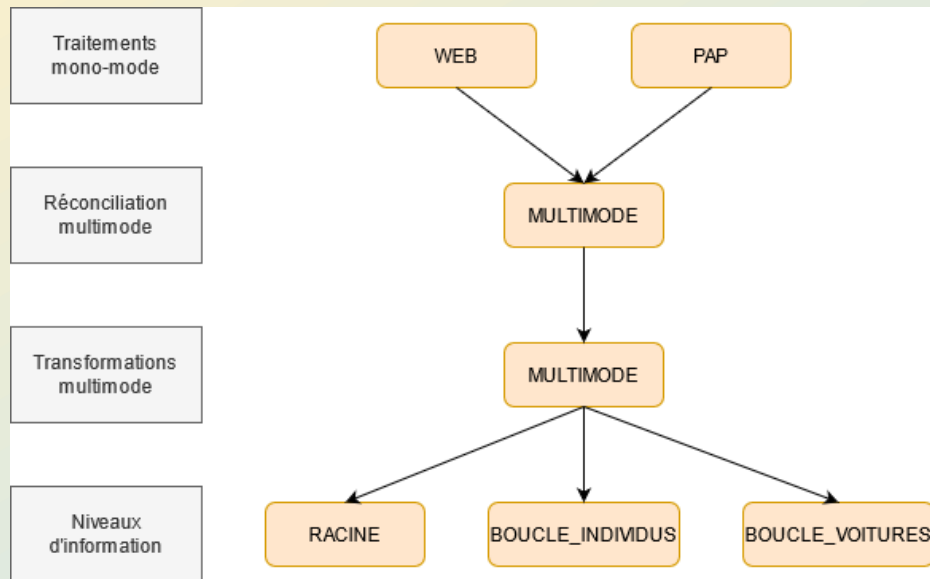
Nécessité de gérer les enquêtes de façon plus efficace (enquêtes infra-annuelles avec des périodes de collecte plus courtes donc 12h de retard = risques accrus)



Un couplage fort avec VTL

- À plusieurs étapes
- Par l'utilisateur ou automatique
- Variables calculées

> des utilisateurs qui ne comprennent pas les incohérences collecte/aval



Démarche de construction

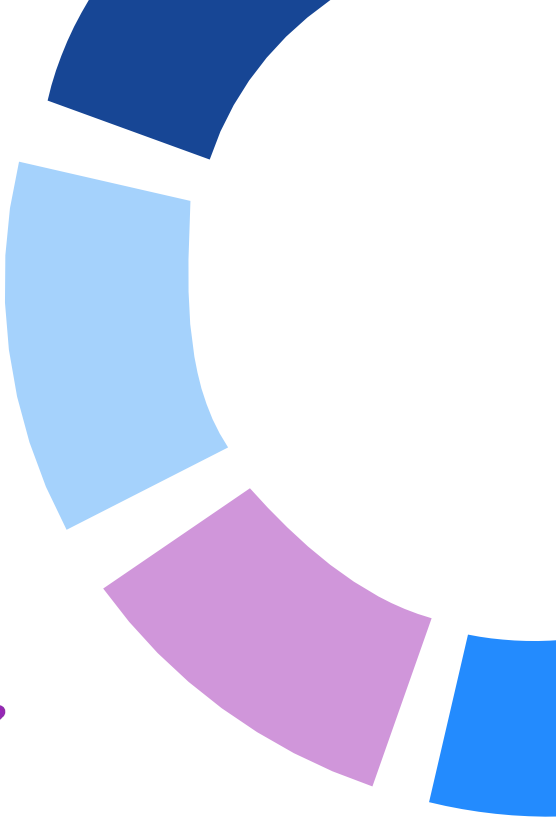
| Pilotée par l'IA (Mistral, puis Claude)

À partir :

- du code (github)
- de la documentation utilisateur (gitlab)

Avec comme objectifs :

- documenter l'existant
- analyser les incohérences code/tests/doc
- dessiner la cible et le chemin pour l'atteindre



02

Architecture cible À valider

Propositions :

Hypothèses de travail

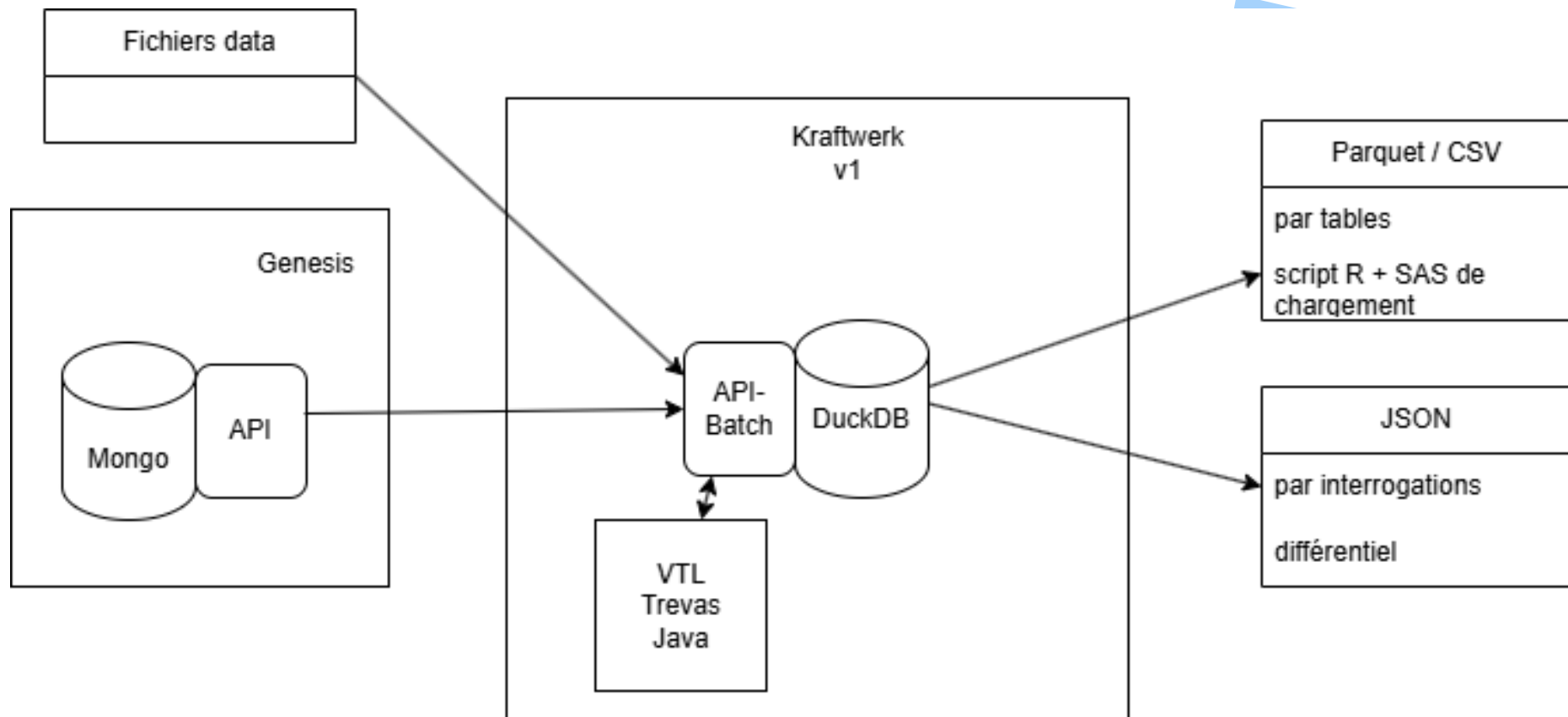
- Suppression de la lecture de données en fichier (base du KW actuel)
- Les *calculated* sont stockées dans Genesis (cf plus loin)
- Les paradata ne sont pas traitées

À valider/discuter :

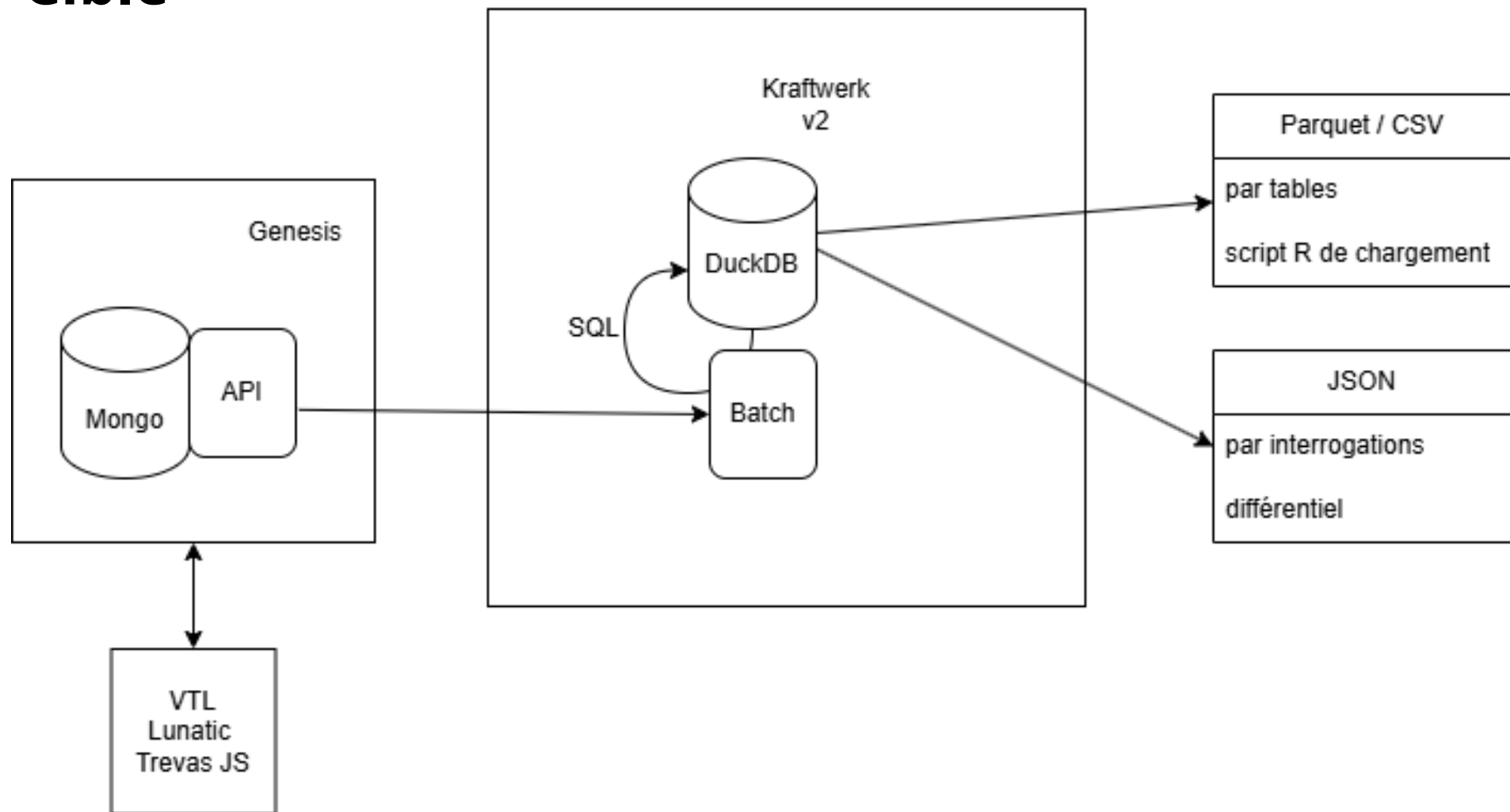
- Faut il un mode API ?
- Intérêt de maintenir la sortie « Reporting Sabiane ? »
- SQL ? Java ? ... R ?
- Utilisation de scripts "aval" pour les exports ?



Actuel



Cible



Zoom VTL - *calculated*

Plusieurs pistes :

- Conserver les calculées dans les PF de collecte ? Bof
- Les calculer en amont et non plus en aval
 - calculées au process et par Perret
 - stockées dans Genesis
 - Envoyées à KW au même format que les collected, external.

À valider :

- Collection à part ?
- Pas de stockage de l'historique (recalculable)
- Pas de nécessité de calcul en mode batch



Zoom VTL - *Traitements à la demande*

Constats actuels :

- fonctionnel proposé historiquement par le POC initial
- au final : peu utilisé, ou alors pour faire de « l'anonymisation » (suppression de colonnes) ou du rattrapage de données mal calculées par VTL

Proposition :

- SQL standard pour supprimer des colonnes (anonymisation)

À valider :

- Nécessité de passer un script personnalisé ? SQL ? R ?



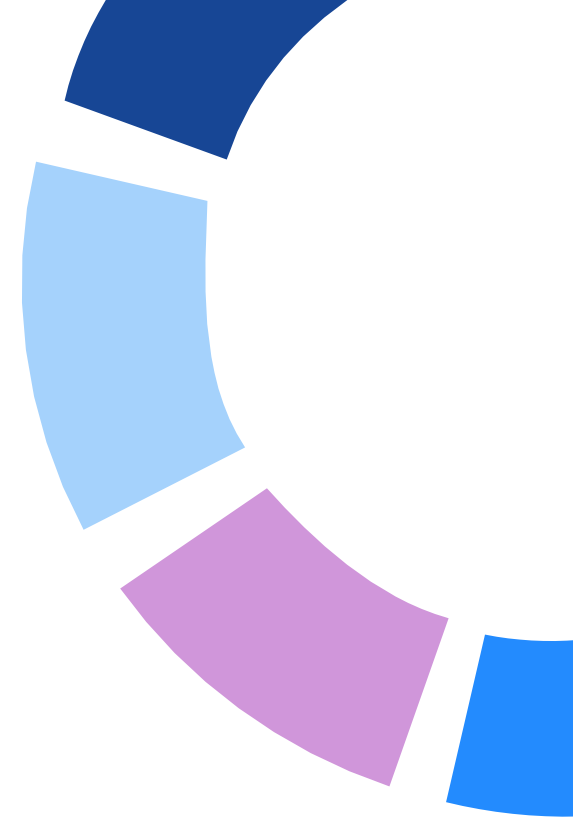
Zoom VTL - *Traitements automatiques*

Autres traitements VTL

- Éclatement en tables => SQL d'insertion
- Liens 2 à 2 déjà calculés en Java (BPM)

À valider :

- ?



Ce que la refonte change

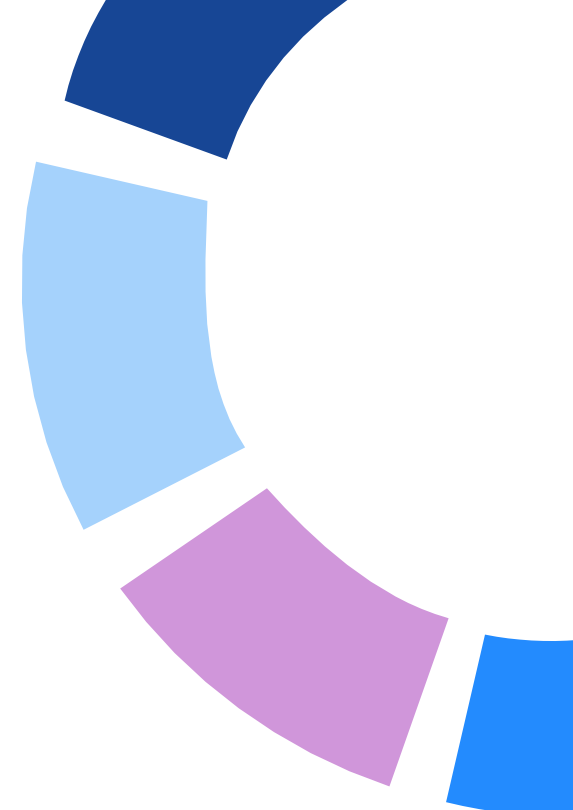
🚫 Disparaît	♻️ Externalisé	🌟 Nouveau / amélioré	✅ Ne change pas
Moteur VTL/Trevas	Calcul des variables → Genesis	partitionId (unité d'appel/export)	Mission : export CSV/Parquet/JSON
Parsing local (XML/Coltrane, <code>XformsDataParser</code>)	Liens 2à2 → BPM	Post-script SQL/R sur tables DuckDB	Gestion des boucles
Paradonnées	Modules TCM → BPM	Anonymisation formalisée	Reporting de collecte
Scripts d'import SAS		Chiffrement bout-en-bout + interchangeable	Stack Java 25 / Spring Boot 4
Archivage des entrées		Suivi de job « au fil de l'eau » unifié	
Endpoints legacy (<code>/main</code> , <code>/steps</code> , <code>/split</code>)		Gestion d'erreurs centralisée ; séparation API/Batch	

03

Roadmap Pas de big bang

Versions prévues

-  V0 – Fondations techniques (non livrable en production)
-  **V1 – Export Json depuis Genesis par partition (MVP enquêtes de conjoncture)**
-  V2 – Exports CSV et parquet, variables d'états, liens 2 à 2
-  V3 – API (si besoin), reporting data, multimode
-  V4 – Chiffrement et anonymisation
-  V5 – Évolutions futures (codification, export à date, ...)



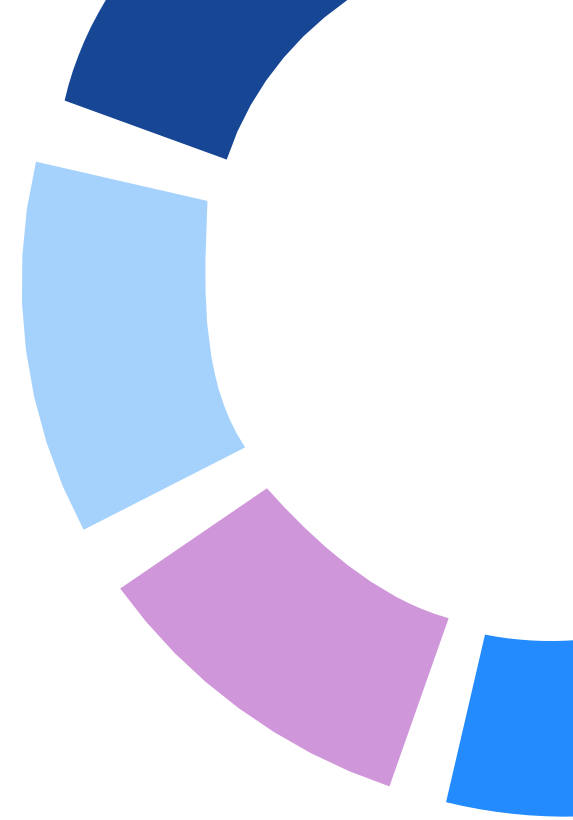
Impact sur les autres briques

BPM : Gérer les liens 2 à 2 (*déjà fait ?*)

Genesis :

- Endpoint /partitions/{partitionId} (*nécessaire même sans refonte*)
- Module VTL JS à intégrer dans le process pour les *calculated* (*nécessaire MVP*)

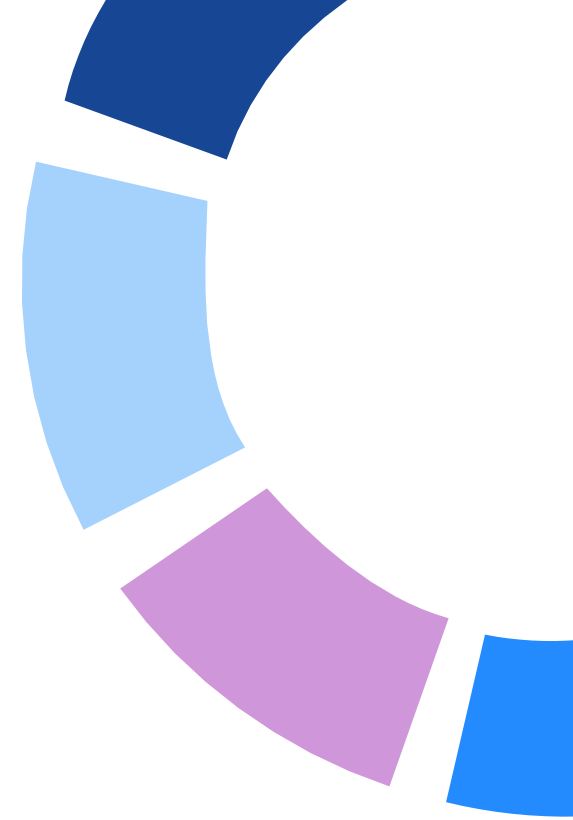
Perret : Enregistrement des *calculated* dans Genesis



Estimations - existant

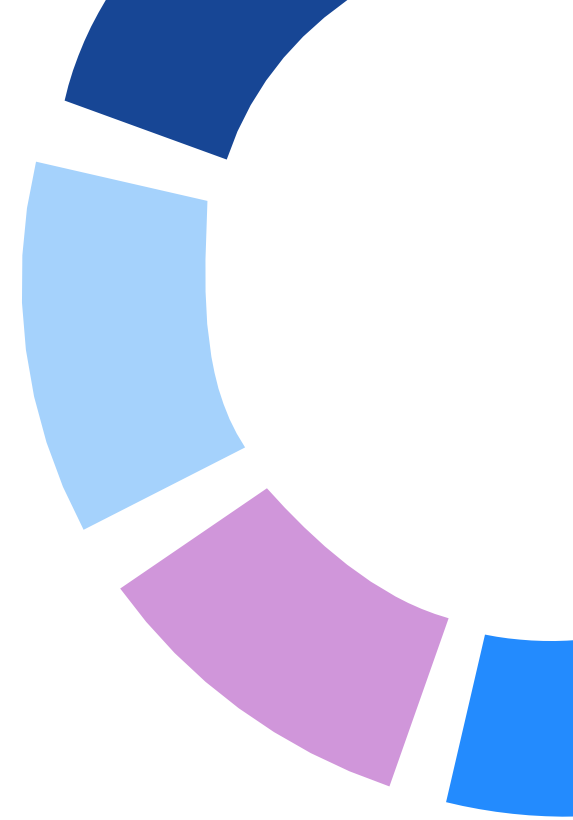
- **Genesis** : Endpoint /partitions/{partitionId} - 13
- **Krafwerk actuel** : adapter au partitionId – 1 sprint

- ✗ Augmentation du coût de maintenance / run
- ✗ Pas de cohérence aval des calculs VTL (sum, first, filter, ...)
- ✓ Pas d'investissement supplémentaire
- ⚠ Coût d'adaptation pour la codification élevé



Estimations - refonte

- **Genesis** : Endpoint /partitions/{partitionId} - 13
- **Genesis** Module VTL JS à intégrer dans le process pour les *calculated* – 13 + 5 de création du module
- **Kraftwerk** 🏗️ **V0** + ⭐ **V1** – Export Json depuis Genesis par partition – 4 sprints (1série)
- **Kraftwerk** 🚀 **V2+3+4** – Iso-fonctionnel – 4 sprints (1série)
 - ❌ Investissement de 2 séries
 - ✅ Cohérence aval des calculs VTL (sum, first, filter, ...)
 - ✅ Coût d'adaptation pour la codification très réduit



Claude Mistral

Développeur expérimenté

Service IA

Insee^{nl}
Mesurer pour comprendre

Merci

insee.fr      [blog](#)

Estimations – refonte fine

- **Genesis** : Endpoint /partitions/{partitionId} - 13
- **Genesis** Module VTL JS à intégrer dans le process pour les *calculated* et stockage – 13
- *Module VTL* : 5
- **Kraftwerkv2** 🏗️ **V0** – Fondations techniques - 13
- **Kraftwerkv2** ⭐ **V1** – Export Json depuis Genesis par partition – 3 sprints
- **Kraftwerkv2** ⭐ **V2+3+4** – 4 sprints
- Perret pour les *calculated* : 8 (3front +5back)

